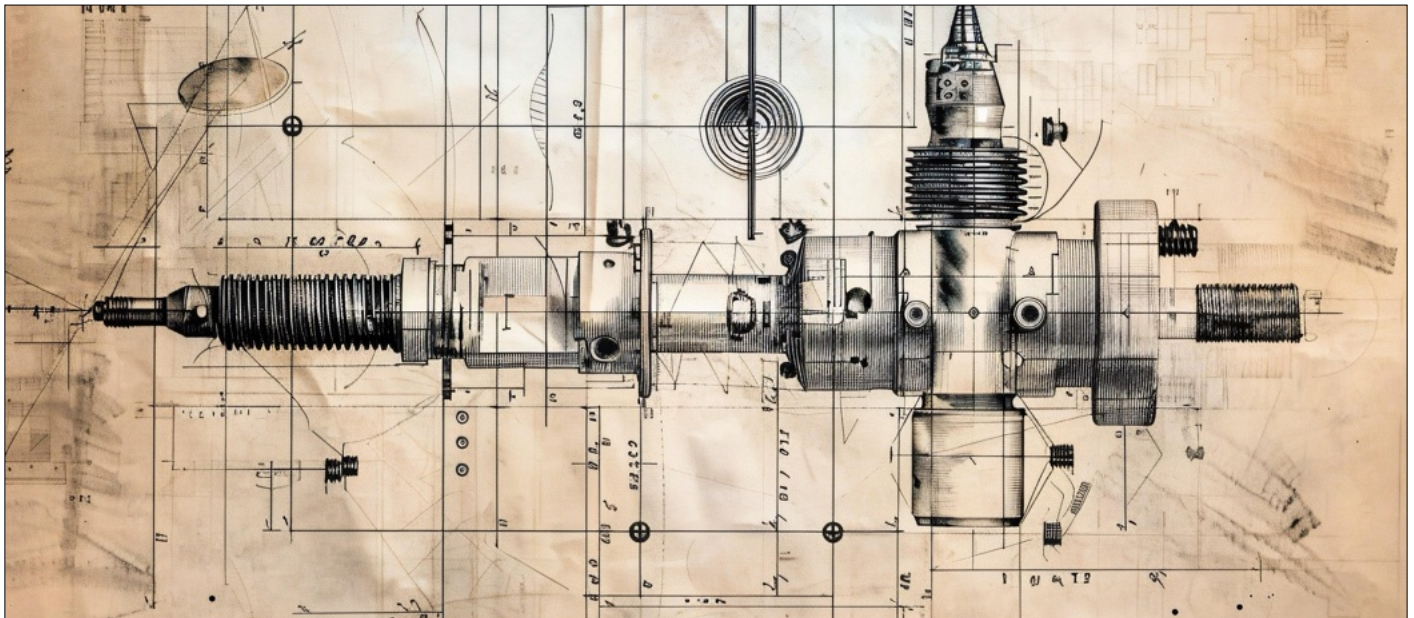




Validating Requirements of ML pipelines using jPipe

S. Mosser, K. Thuan-Phong Khuu
2026-08-17 — Part II
IEEE RE'26

1

Understanding “Decision Drift”

2

jPipe & Justification Models

3

Modelling a Justification

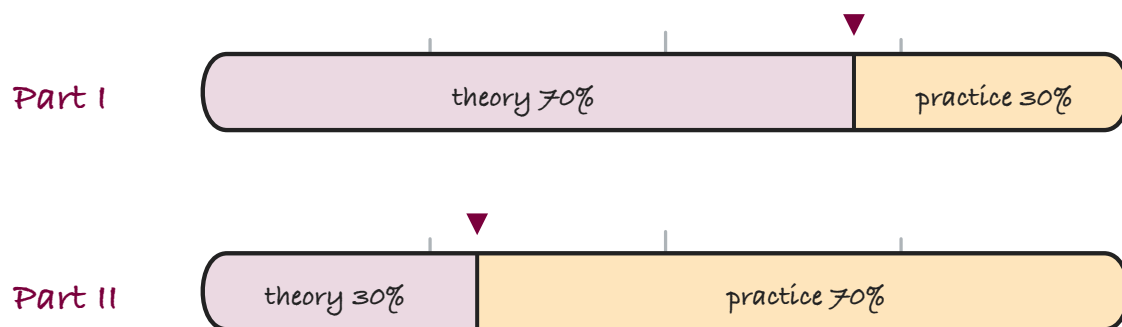
4 Executing Justifications

Composing at Scale

6 Wrap up & Key Takeaways

5

Overall Organization

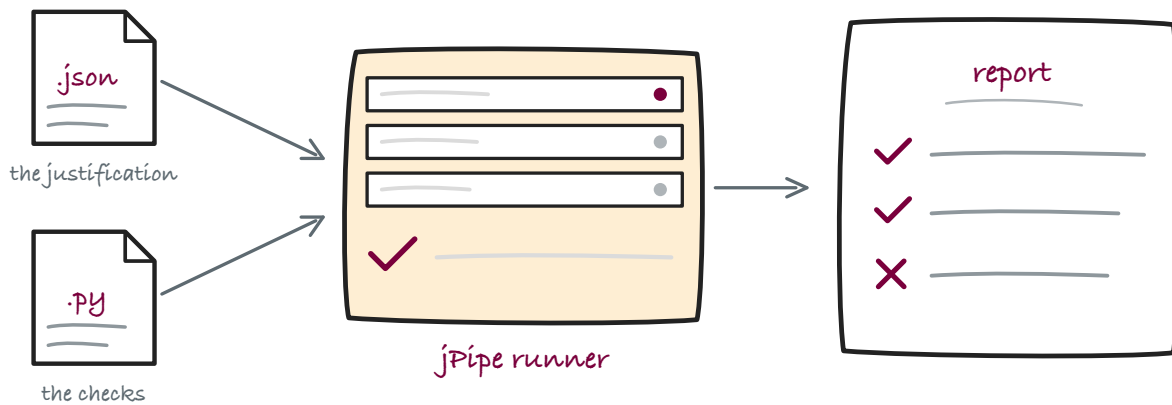


Executing Justifications

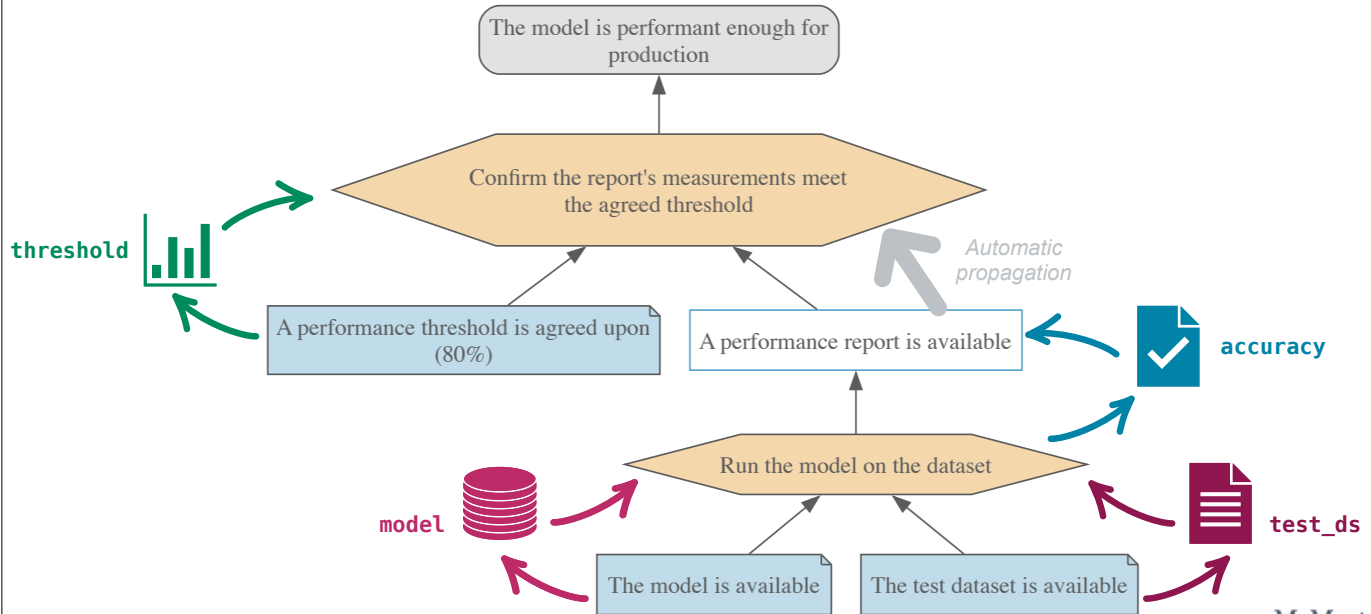
Let's validate that we're meeting our requirements!

4

jPipe Runner Architecture



From “argument” to “data flow”



```

def the_test_dataset_is_available(producer: JpipeProducer) -> bool:
    52 producer("dataset", "test-split")
    53 return True
    54
    55
    56 @pipe.link("deployable:execution")
    57 @pipe.producer("measured_accuracy", consumer=("trained_model", "dataset"))
    58 def run_the_model_on_the_dataset(trained_model, dataset,
    59                                 producer: JpipeProducer) -> bool:
    60     """[strategy] Run the model on the dataset"""
    61     # A run needs both of the things below it, and produces the measurement the argument
    62     # is about. It reads no bar: whether the number is good enough is not its call.
    63     if not (trained_model and dataset):
    64         return False
    65     producer("measured_accuracy", MEASURED_ACCURACY)
    66     return True
    67
    68
    69 @pipe.link("deployable:threshold")
    70 @pipe.consumer("measured_accuracy", "agreed_bar")
    71 def confirm_the_report_s_measurements_meet_the_agreed_threshold(measured_accuracy,
    72                                                                agreed_bar) -> bool:
    73     """[strategy] Confirm the report's measurements meet the agreed threshold"""
    74     # The one judgement in the argument: a measured number, and the bar it was agreed
    75     # against, each arriving from the element that establishes it.
    76     return measured_accuracy == agreed_bar
    77
    78 # The runner passes this claim through unless it is bound. Be comment to check it.
    79
  
```

Problems Output Debug Console Terminal Ports

(tutorial-re-2026) messenger@pegar tutorial-re-2026 % jpipe-runner -l part_3/chapter_3/deployable.py part_3/chapter_3/deployable.json

JPIPE Runner

JPIPE Files

JPIPE Files Justification :: deployable

evidence=deployable:agreed	:: A performance threshold is agreed u...	PASS
evidence=deployable:measured_accuracy	:: The model is available	PASS
evidence=deployable:test_ds	:: The test dataset is available	PASS
strategy=deployable:execution	:: Run the model on the dataset	PASS
sub-conclusion=deployable:reporting	:: A performance report is av...	PASS
strategy=deployable:threshold	:: Confirm the report's measurement...	PASS
conclusion=deployable:claim	:: The model is performant enough for...	PASS

JPIPE Files

Justification, 7 passed, 0 failed, 0 skipped

(tutorial-re-2026) messenger@pegar tutorial-re-2026 %

An Evidence must produce “something”

Bind a function to a model element

Declare the produced artefact(s)

```
@jpipe_link("deployable:model")
@jpipe(produce=["trained_model"])
def the_model_is_available(produce: JpipeProduce) -> bool:
    """[evidence] The model is available"""
    # Real code goes here (file exists, load file, ...)

    produce("trained_model", "emotion-classifier-c")

    return True
```

Does the element validate?

Register the artefact

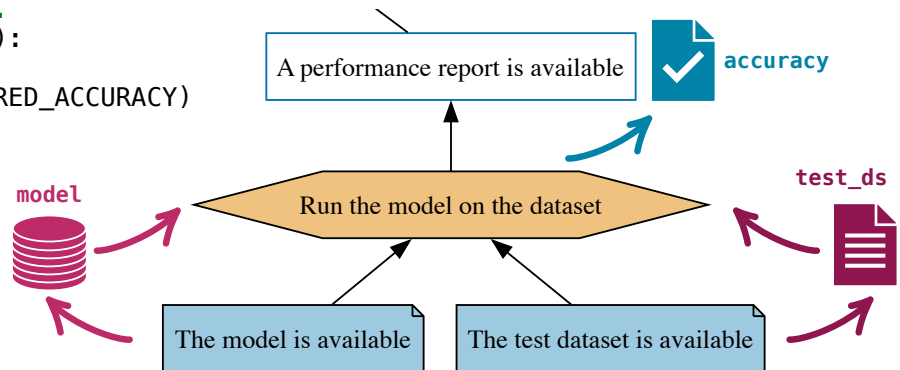
The model is available
deployable:model

trained_model



A strategy consumes and (might) produce

```
@jpipe_link("deployable:execution")
@jpipe(produce=["accuracy"], consume=["model", "test_ds"])
def run_the_model_on_the_dataset(model, test_ds,
                                produce: JpipeProduce) -> bool:
    """[strategy] Run the model on the dataset"""
    # Real run code goes here.
    if not (model and test_ds):
        return False
    produce("accuracy", MEASURED_ACCURACY)
    return True
```



```

@jpipe_link("deployable:execution")
@jpipe(produce=["measured_accuracy"], consume=[])
def run_the_model_on_the_dataset(trained_model, dataset,
                                produce: JpipeProduce) -> bool:
    """[strategy] Run the model on the dataset"""
    if not (trained_model and dataset):
        return False
    produce("measured_accuracy", MEASURED_ACCURACY)
    return True

```

One must consumes what is produced upstream

WARNING - validate!1316 - 2026-08-15 20:32:55,982 - [ProducedButNotConsumedValidator]
 Pipeline validation error: produced variable not consumed.
 • Variable 'trained_model' is produced by function 'the_model_is_available' but is never consumed by any function.
 • This may indicate redundant computation or misconfiguration.
 • Consider removing the production of 'trained_model' if unused, or verify downstream usage.
 [ProducedButNotConsumedValidator]
 Pipeline validation error: produced variable not consumed.
 • Variable 'dataset' is produced by function 'the_test_dataset_is_available' but is never consumed by any function.
 • This may indicate redundant computation or misconfiguration.
 • Consider removing the production of 'dataset' if unused, or verify downstream usage.
 ERROR - validate!1318 - 2026-08-15 20:32:55,982 - [EvidenceDependencyValidator]
 Pipeline validation error: leaf node variables not consumed by connected strategies.
 • Element: deployable:model ("the model is available")
 • Produced variables: ["dataset"]
 • Affected strategies: ["run_the_model_on_the_dataset"]
 • Problem: These strategies do not consume all variables produced by deployable:model.
 • Fix: Verify that the strategies listed above declare consume[...] for all outputs of deployable:model.
 [EvidenceDependencyValidator]
 Pipeline validation error: leaf node variables not consumed by connected strategies.
 • Element: deployable:test_ds ("the test dataset is available")
 • Produced variables: ["dataset"]
 • Affected strategies: ["run_the_model_on_the_dataset"]
 • Problem: These strategies do not consume all variables produced by deployable:test_ds.
 • Fix: Verify that the strategies listed above declare consume[...] for all outputs of deployable:test_ds.

11

```

@jpipe_link("deployable:execution")
@jpipe(produce=["measured_accuracy"], consume=["trained_model", "dataset"])
def run_the_model_on_the_dataset(trained_model, dataset,
                                produce: JpipeProduce) -> bool:
    """[strategy] Run the model on the dataset"""
    # if not (trained_model and dataset):
    #     return False
    produce("measured_accuracy", MEASURED_ACCURACY)
    return True

```

Validation rules help ensuring the Python code is "reasonably aligned" with the argument structure

Error loading 'part_3/chapter_2/deployable.py':
 [JupyterLab] Function 'run_the_model_on_the_dataset' declares consumed variables: ['dataset', 'trained_model'] that are never referenced in the function body.
 • Fix: either reference each variable in the function body, or remove it from consume[].
 (tutorial-re-2026) messenger@pghpghor tutorial-re-2026 %

12

Validation report

○ (tutorial-re-2026) mosser@belphegor tutorial-re-2026 % jpipe-runner -l part_1/ch

jPipe Runner

```
=====
jPipe Files
=====
jPipe Files.Justification :: deployable
=====
evidence<deployable:agreed> :: A performance threshold is agreed u... | PASS |
evidence<deployable:model> :: The model is available | PASS |
evidence<deployable:test_ds> :: The test dataset is available | PASS |
strategy<deployable:execution> :: Run the model on the dataset | PASS |
sub-conclusion<deployable:reporting> :: A performance report is av... | PASS |
strategy<deployable:threshold> :: Confirm the report's measurement... | FAIL |
conclusion<deployable:claim> :: The model is performant enough for... | SKIP |
=====
jPipe Files
1 justification, 5 passed, 1 failed, 1 skipped
=====
```

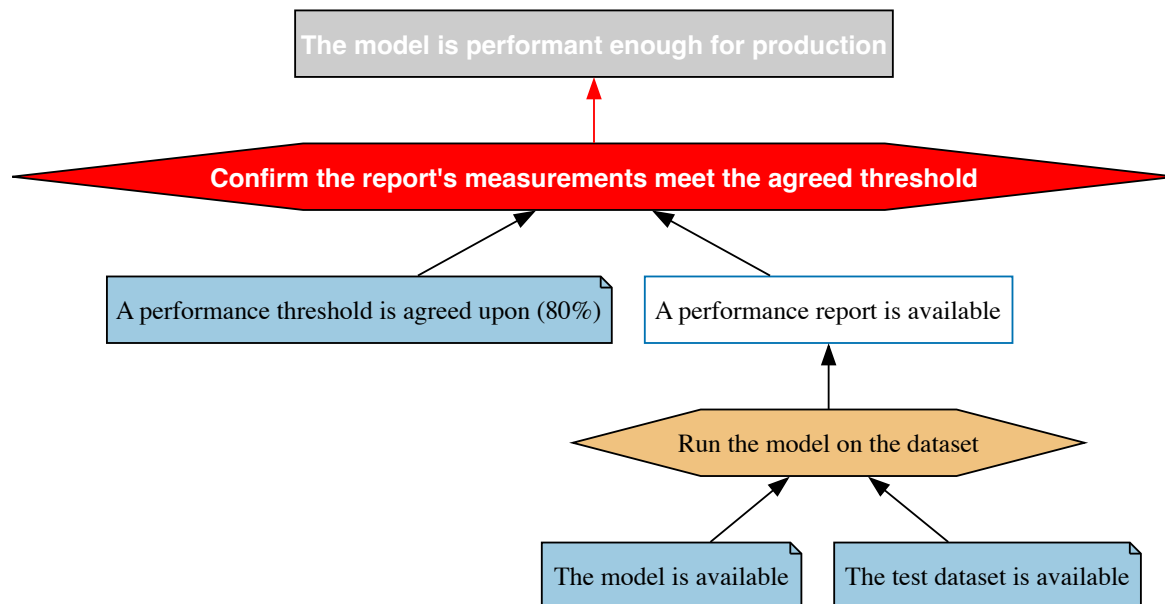
● (tutorial-re-2026) mosser@belphegor tutorial-re-2026 % jpipe-runner -l part_1/ch

jPipe Runner

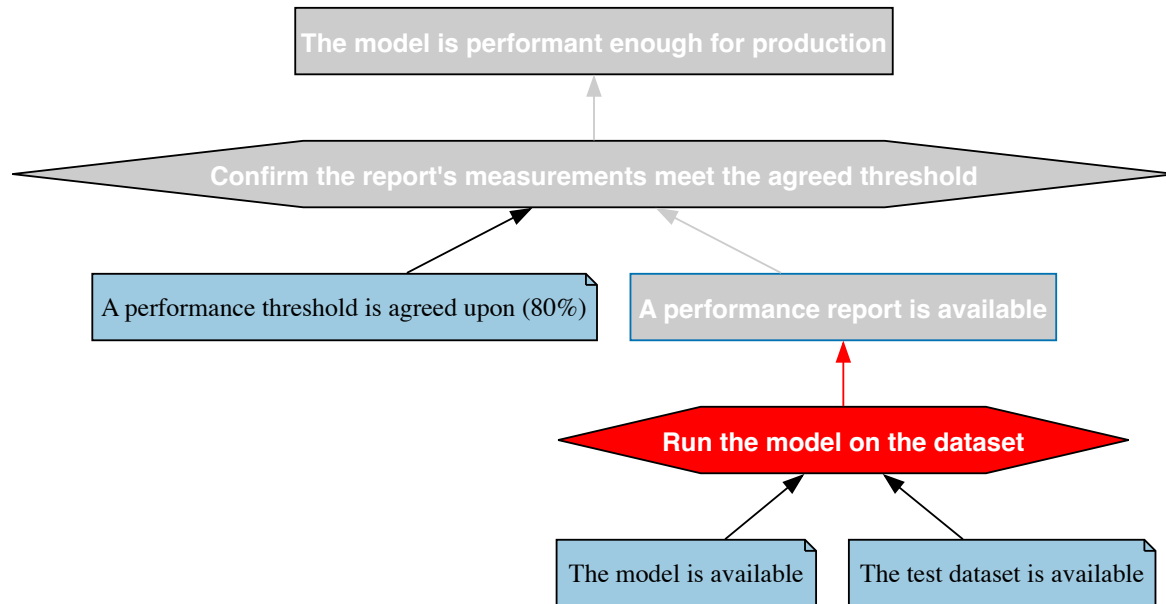
```
=====
jPipe Files
=====
jPipe Files.Justification :: deployable
=====
evidence<deployable:agreed> :: A performance threshold is agreed u... | PASS |
evidence<deployable:model> :: The model is available | PASS |
evidence<deployable:test_ds> :: The test dataset is available | PASS |
strategy<deployable:execution> :: Run the model on the dataset | PASS |
sub-conclusion<deployable:reporting> :: A performance report is av... | PASS |
strategy<deployable:threshold> :: Confirm the report's measurement... | PASS |
conclusion<deployable:claim> :: The model is performant enough for... | PASS |
=====
jPipe Files
1 justification, 7 passed, 0 failed, 0 skipped
=====
```

○ (tutorial-re-2026) mosser@belphegor tutorial-re-2026 % █

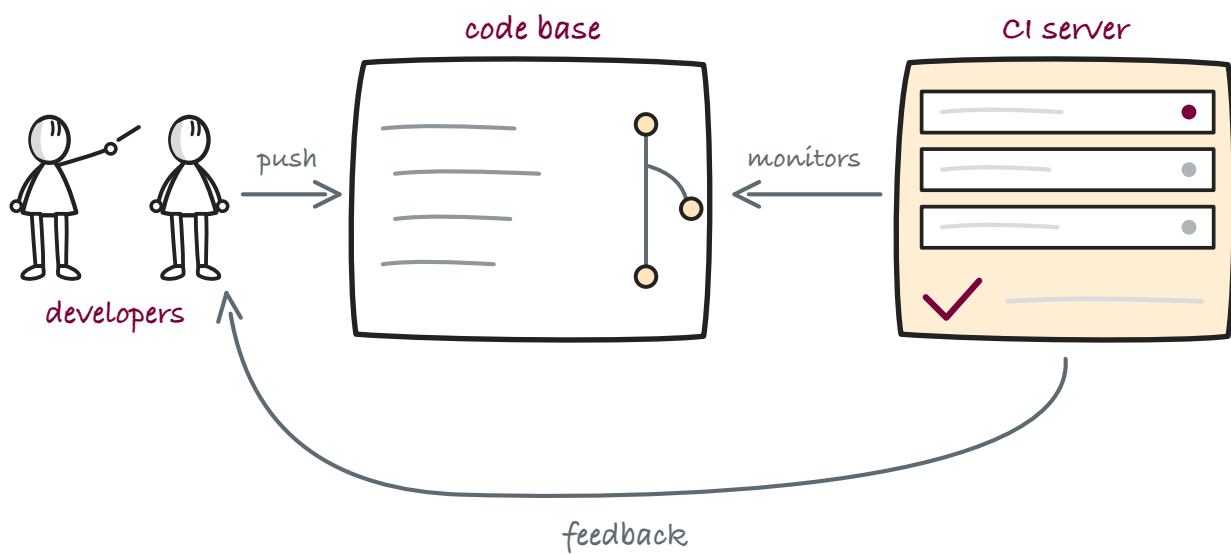
Enriching the diagram with status checks



Non-supported elements are skipped



Continuous Integration



Integrating jPipe with Github Actions



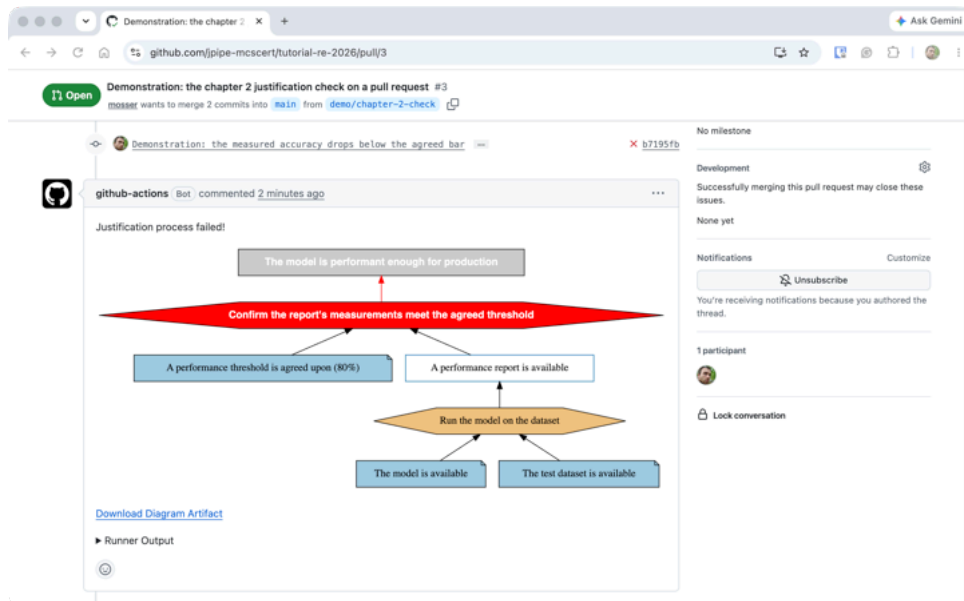
```
jobs:
  deployable:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - name: Check the argument, and draw the verdict
        uses: jpipe-mcscert/jpipe-runner@v3.5.3
        with:
          version: v3.5.3
          jd_file: part_1/chapter_2/deployable.json
          library: part_1/chapter_2/deployable.py
          embed_image: "true"
          image_branch: "diagram-images"
          github-token: ${ secrets.GITHUB_TOKEN }}
```

Seamless integration (DevOps/MLOps)

The screenshot shows the GitHub Actions interface for the repository 'jpipe-mcscert / tutorial-re-2026'. The 'Actions' tab is selected, and the workflow 'Justification (chapter 2)' is displayed. The workflow has two runs:

Run	Status	Event	Branch	Actor	Time
1	Success	Demonstration: the chapter 2 justification che...	demo/chapter-2-check	mosser	Aug 14, 8:34 PM EDT 46s
2	Failure	Demonstration: the chapter 2 justification che...	demo/chapter-2-check	mosser	Aug 14, 8:33 PM EDT 41s

Report: comment(s) on Pull Request(s)



Time to execute your first models!



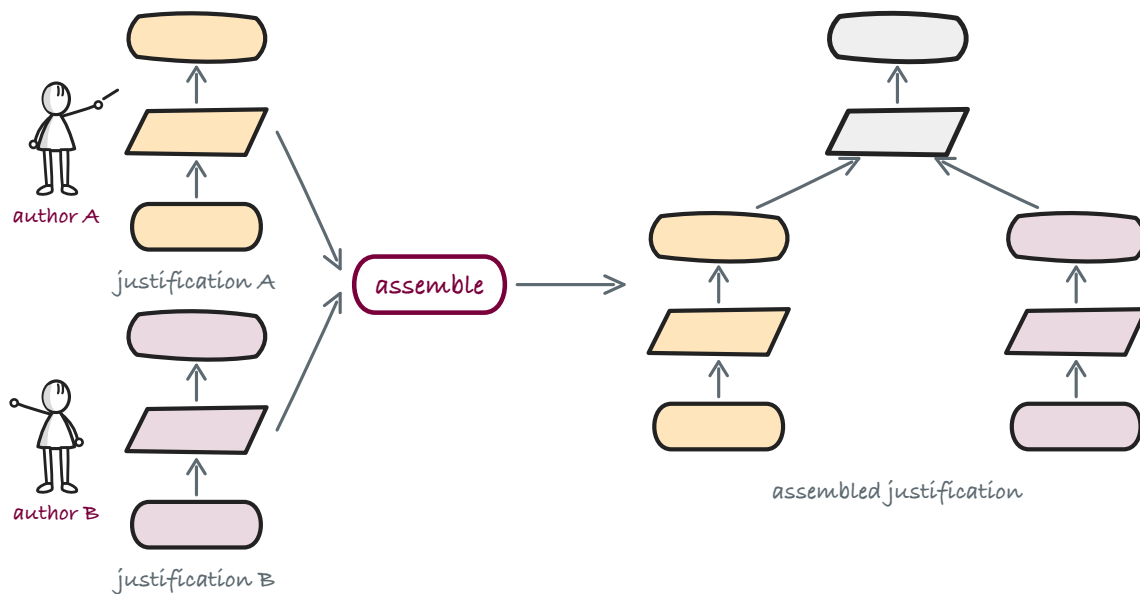
- **Open your Codespace** or local IDE
 - If not already: <https://bit.ly/code-re2026>
- Navigate to **part_2 >> chapter_4 >> README.md**
- **Tasks**
 1. **Scaffold a Python skeleton** for a justification.
 2. **Implement** evidence(s) and strategy **code**
 3. **Execute it** to validate your claim
- **Followed by: Live demo** of CI/CD integration

Composing at scale

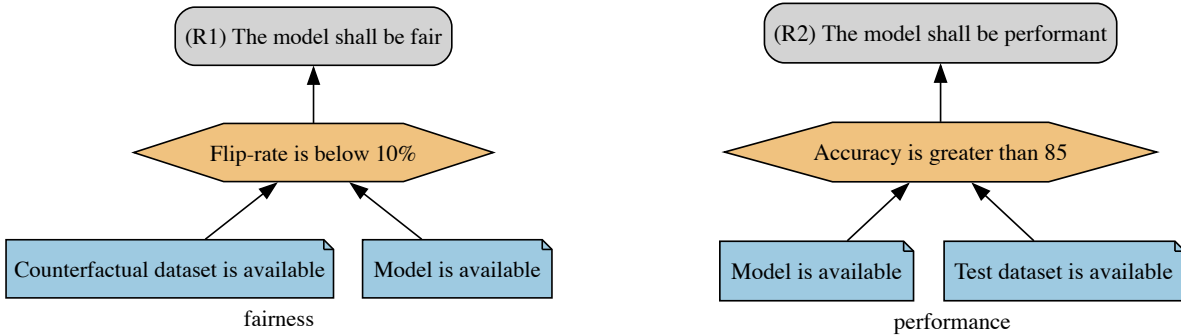
Size matters.

5

Assembling sub-goals into higher ones



Separation of concerns: Fairness & Perf.

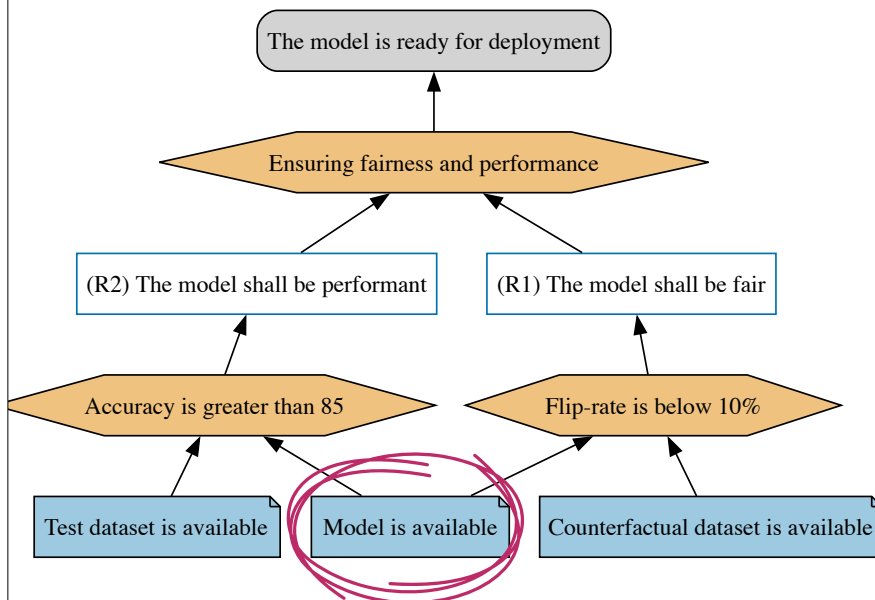


```

justification deployable is assemble(performance, fairness) {
  conclusionLabel: "The model is ready for deployment"
  strategyLabel: "Ensuring fairness and performance"
}

```

Equivalent elements are unified



JPIPE Preview

Problems 0 Models 3 Symbols 17 Actions 38 Copy Report Text JSON

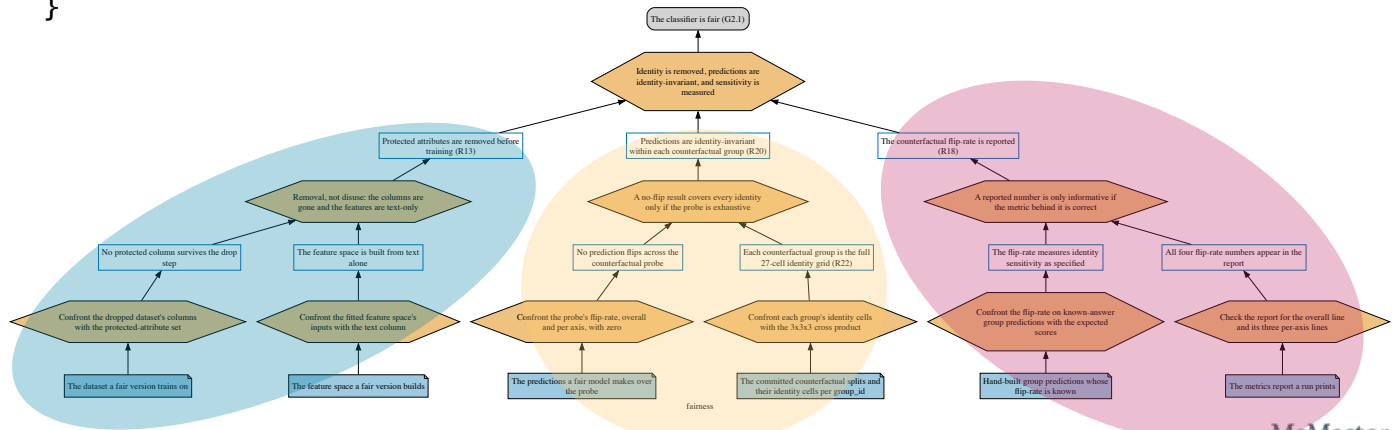
Filter...

fairness justification			
fair	conclusion		2:15
flip	strategy		3:15
counterfactual	evidence		4:15
model	evidence		5:15
performance justification			
performant	conclusion		13:15
accuracy	strategy		14:15
model	evidence		15:15
test	evidence		16:15
deployable justification			
assembleConclusion	conclusion	SYNTHESIZED	22:14
performance:performant	sub-conclusion		13:15
fairness:fair	sub-conclusion		2:15
performance:accuracy	strategy		14:15
fairness:flip	strategy		3:15
assembleStrategy	strategy	SYNTHESIZED	22:14
unified_0	evidence		15:15
performance:test	evidence		16:15
fairness:counterfactual	evidence		4:15
performance:model	→ unified_0	alias	
fairness:model	→ unified_0	alias	

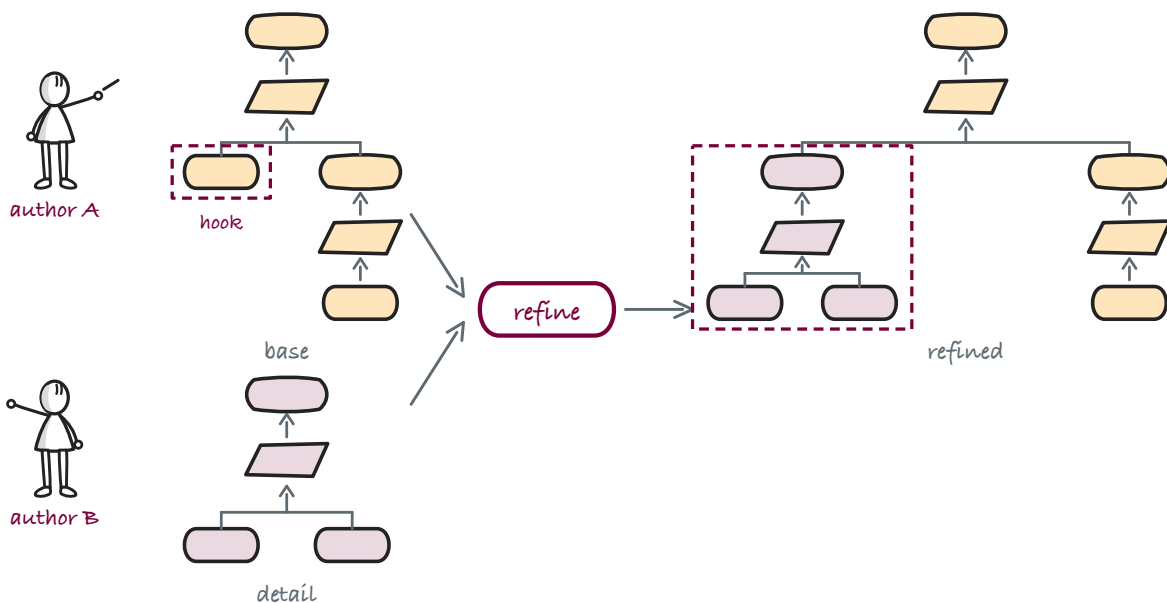
Emotion detection fairness requirements

```
load "requirements/r{13,18,20}.jd"
```

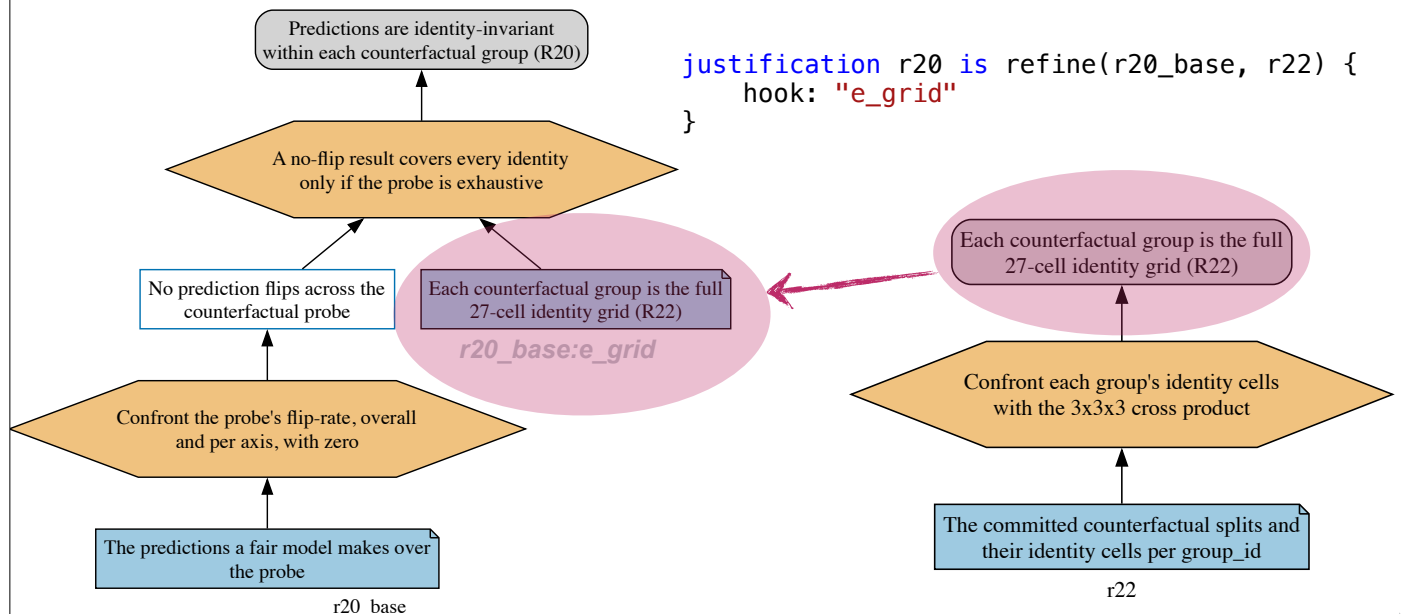
```
justification fairness is assemble(r13, r20, r18) {
  conclusionLabel: "The classifier is fair (G2.1)"
  strategyLabel:  "Identity is removed, ..."
}
```



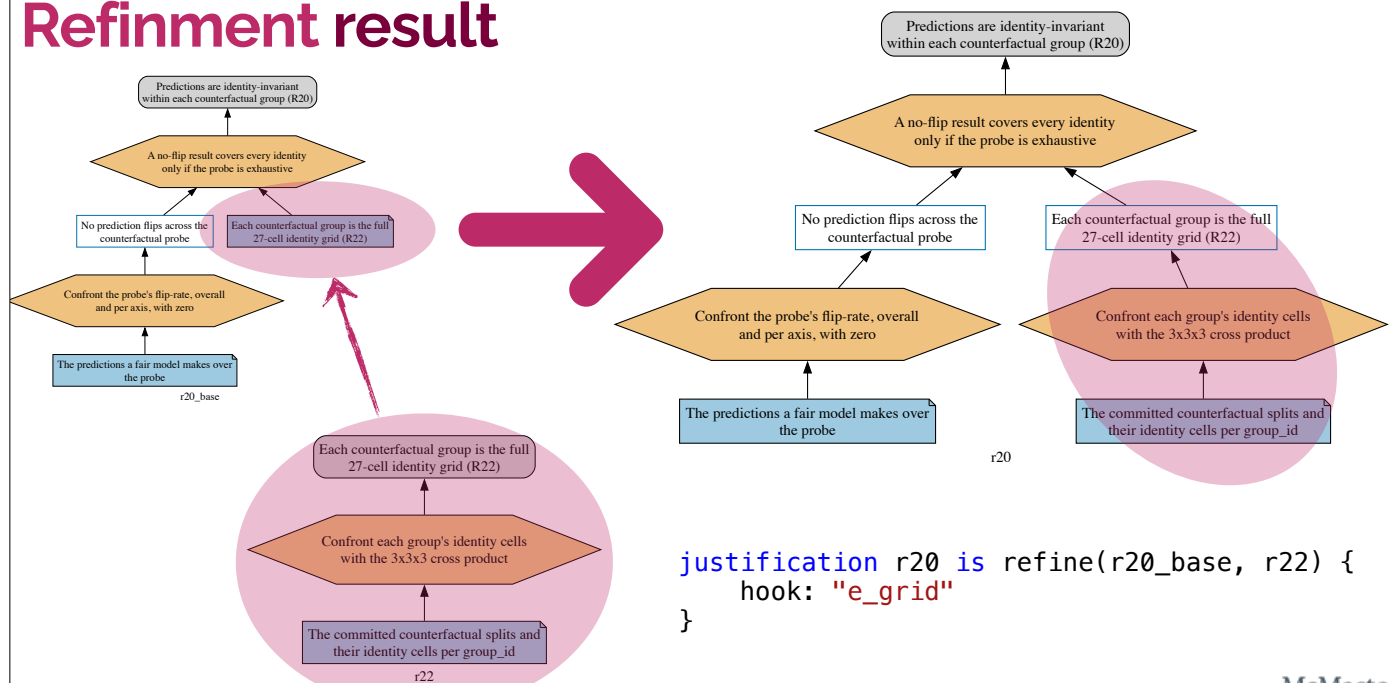
Refining evidence with sub-claims



Refine is a Syntactic substitution



Refinement result

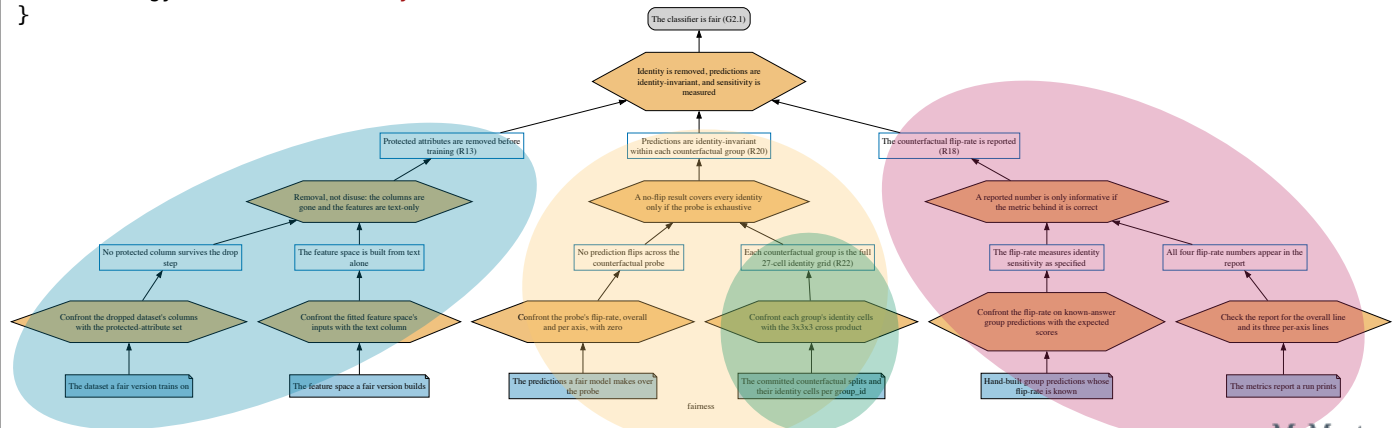


Composition operators are composable

```
justification r20 is refine(r20_base, r22) {
  hook: "e_grid"
}

justification fairness is assemble(r13, r20, r18) {
  conclusionLabel: "The classifier is fair (G2.1)"
  strategyLabel: "Identity is..."
}
```

Declaration order does not matter
(resolved at compilation time)



Time to compose your own models!



- **Open your Codespace** or local IDE
- If not already: <https://bit.ly/code-re2026>
- Navigate to **part_1 >> chapter_5 >> README.md**

• Tasks

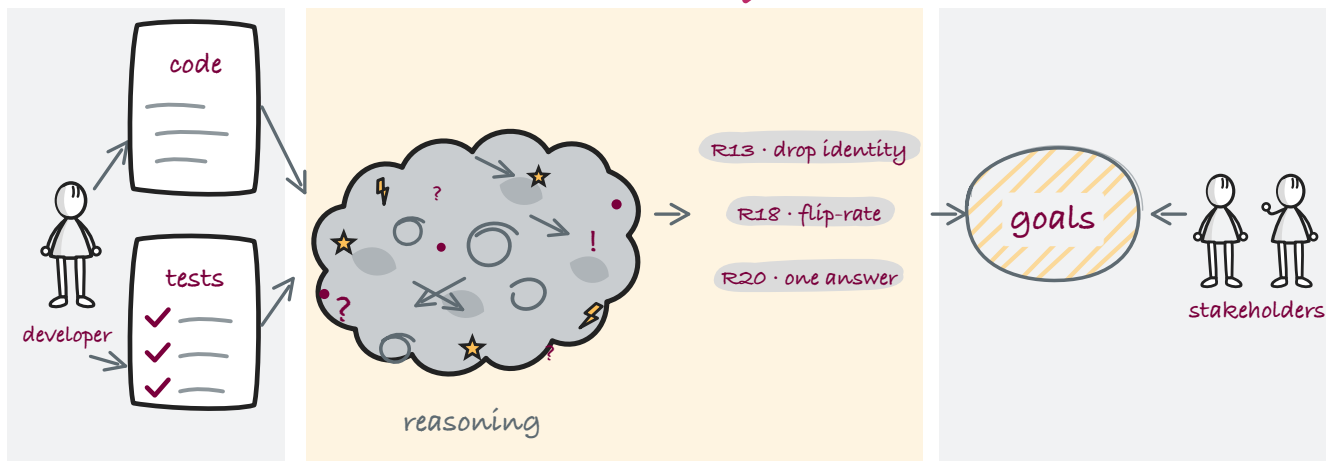
1. **Load** and **Assemble** justifications
2. **Refine** a justification with another one
3. **Compose** calls to assemble and refine.

Wrapup & Key Takeaways

It's already time to leave...

6

jPipe in a nutshell



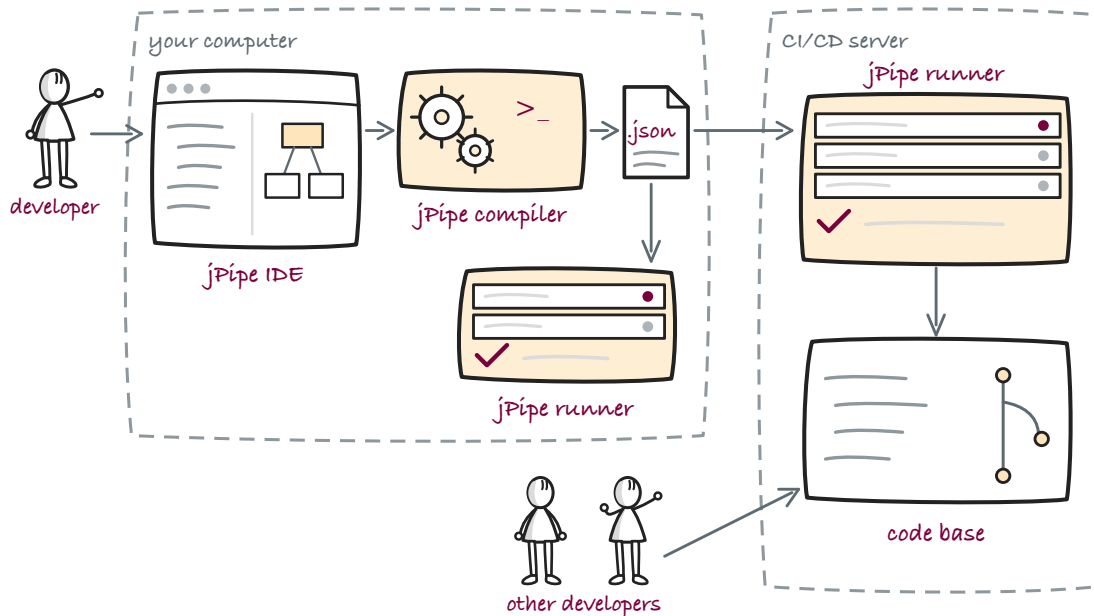
We align with developers' practices (Python code, tests)

We offer **jPipe**, a way to **model the justification**, **operationalize it**, and **integrate it** to avoid decision drift

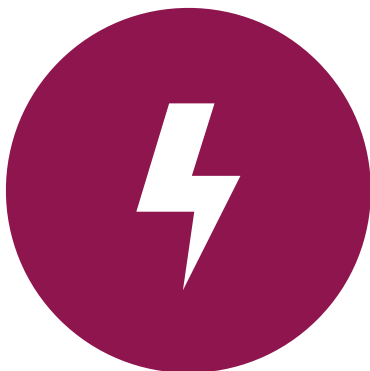
We work with **what we have** (and incrementally add what we'll discover later)



A Tool Ecosystem to support validation



Summary



- **jPipe** allows one to write **justification models**
 - Make **claims** to support **Requirements Validation**
- **Toulmin-like arguments** that can be:
 - **Executed** (locally, on a CI/CD pipeline)
 - **Composed** (to build arbitrarily complex ones)
- **A Tool Ecosystem:** *Compiler, IDE, Runner*
 - Available as an **open-source stack** (MIT License)
 - **Open for collaborations** & new users!

Time for discussions!



- *Proofs, or Evidence?*
- *Requirements? Design Decision? Assumption?*
- *When is an argument enough?*
- *Can all requirements be justified?*
- *Requirements & justification co-evolution*
- *Ownership when assembling/refining*
- *Role of justifications in external audit*



ENGINEERING

Computing
& Software



McSCert



Dr. Sébastien Mosser
Associate Professor
mossers@mcmaster.ca



Calvin Thuan-Phong Khuu
PhD Student
khuu2@mcmaster.ca